QuarterExaminerViewer

An SoQtExaminerViewer-style viewer for Coin3D, built on Quarter

Overview

`QuarterExaminerViewer` is a Qt widget that gives applications built on the Quarter library the look, feel and functionality of the classic `SoQtExaminerViewer`. Quarter supplies the OpenGL canvas; `QuarterExaminerViewer` adds the viewer decoration (buttons and thumbwheels), mouse navigation, a popup menu, a preferences dialog, an animated seek function, a home position and perspective/orthographic camera switching.

The class derives from `QWidget`, so it can be used as a top-level window or embedded in any layout. The underlying `QuarterWidget` is available through `quarterWidget()` for anything the class does not wrap.

Defaults

- Starts in View mode (the mouse navigates the camera).
- Headlight on; black background; decoration visible.
- Seek: 2.0 seconds, 50 % of the current distance, seeks to the picked point.
- Right-click popup menu enabled.
- Default window size 640 x 520 pixels.

Synopsis

```
#include "QuarterExaminerViewer.h"

class QuarterExaminerViewer : public QWidget
{
    Q_OBJECT
public:
    enum CameraType { Perspective, Orthographic };
    enum DrawStyle  { AsIs, Wireframe, WireframeOverlay,
                      HiddenLine, Points, BoundingBox };
    enum TransparencyType { NoTransparency, ScreenDoor, Add, DelayedAdd,
                            SortedObjectAdd, Blend, DelayedBlend,
                            SortedObjectBlend };

    explicit QuarterExaminerViewer(QWidget* parent = nullptr);
    ~QuarterExaminerViewer();
    ...
};
```

Quick start

```
#include <QApplication>
#include <Quarter/Quarter.h>
#include <Inventor/nodes/SoCone.h>
#include <Inventor/nodes/SoMaterial.h>
#include <Inventor/nodes/SoSeparator.h>
#include "QuarterExaminerViewer.h"

int main(int argc, char** argv)
{
    QApplication app(argc, argv);
    SIM::Coin3D::Quarter::Quarter::init();    // after QApplication

    SoSeparator* root = new SoSeparator;
    root->ref();
    SoMaterial* mat = new SoMaterial;
    mat->diffuseColor.setValue(1.0f, 0.0f, 0.0f);
    root->addChild(mat);
    root->addChild(new SoCone);               // no camera: the viewer adds one

    {
        QuarterExaminerViewer viewer;
        viewer.setSceneGraph(root);
        viewer.show();
        app.exec();
    }   // the viewer must be destroyed before Quarter::clean()

    root->unref();
    SIM::Coin3D::Quarter::Quarter::clean();
    return 0;
}
```

Building

The supplied `CMakeLists.txt` builds a static library `quarterexaminerviewer` and the `hello_cone` test program. Link your own targets against the library to inherit the include path and the Qt, Coin, Quarter and OpenGL dependencies:

```
target_link_libraries(myapp PRIVATE quarterexaminerviewer)
```

If Quarter and Coin are not found automatically, pass `-DQUARTER_ROOT=<prefix>` and `-DCOIN_ROOT=<prefix>`, or the directories containing their CMake config files. Set `-DQUARTER_QT_MAJOR=5` or `6` if Qt auto-detection chooses a version different from the one Quarter was built with.

Viewer decoration

Buttons (left strip)

Button	Function
Pick	Pick mode: mouse events go to the scene graph (for manipulators, selection, etc.). Arrow cursor.
View	View mode: the mouse navigates the camera. Hand cursor.
Home	Return to the stored home viewpoint.
Set Home	Store the current viewpoint as the home viewpoint.
View All	Fit the whole scene in the window.
Seek	Fly to an object: press, then left-click the target.
Camera	Switch between perspective and orthographic camera. The icon shows the type that a click switches to.

Thumbwheels

Wheel	Location	Function
Rotx	Left strip, bottom	Orbit the camera about its horizontal axis.
Roty	Bottom strip	Orbit the camera about its vertical axis.
Dolly	Right strip	Move the camera towards or away from the scene (or change the view height for an orthographic camera).

All rotations are about the camera's focal point. The decoration can be hidden with `setDecoration(false)` or the popup menu.

Mouse and keyboard

View mode

Input	Action
Left drag	Rotate (orbit about the focal point)
Middle drag	Pan
Ctrl + left drag	Pan
Ctrl + middle drag	Zoom
Shift + left drag	Zoom
Mouse wheel	Zoom (wheel up = closer)
Right click	Popup menu

Keyboard

Key	Action
Esc	Toggle between Pick and View mode

In View mode the mouse belongs to the viewer: no mouse event, hover included, is passed to the scene graph. In Pick mode the mouse events are passed on to Quarter and the scene graph; only the right-click popup menu and the Esc key remain handled by the viewer. The mouse cursor over the canvas is an open hand in View mode (closed while dragging), an arrow in Pick mode and a crosshair while a seek is armed.

Popup menu and preferences

Popup menu (right click)

- `Functions`: Home, Set Home, View All, Seek.
- `Viewing`: toggle View / Pick mode.
- `Draw Style`: As is, Wireframe, Wireframe overlay, Hidden line, Points, Bounding box.
- `Transparency Type`: None, Screen door, Add, Delayed add, Sorted object add, Blend, Delayed blend, Sorted object blend.
- `Orthographic Camera`: toggle camera type.
- `Headlight`, `Decoration`: toggles.
- `Preferences...`: opens the preferences dialog.

Preferences dialog

Setting	Meaning
Seek animation time	Duration of the seek flight in seconds (0 = jump immediately).
Seek distance	Percentage of the current camera-to-target distance kept after a seek (default 50 %). For an orthographic camera this scales the view height.
Seek to	`Picked point` or `Object center` (center of the picked object's bounding box).
Camera zoom	Field of view of a perspective camera in degrees (disabled for an orthographic camera).
Background color	Opens a color chooser.
Transparency type	Quarter transparency mode.
Headlight	Enable or disable the headlight.

Changes take effect immediately; the dialog only has a Close button.

Enumerations

`enum CameraType { Perspective, Orthographic }`

The camera projection. Used by `cameraType()` and `setCameraType()`.

`enum DrawStyle { AsIs, Wireframe, WireframeOverlay, HiddenLine, Points, BoundingBox }`

How the scene is rendered. Maps onto Quarter's render modes.

`enum TransparencyType { NoTransparency, ScreenDoor, Add, DelayedAdd, SortedObjectAdd, Blend, DelayedBlend, SortedObjectBlend }`

How transparent objects are rendered. Maps onto Quarter's transparency types. `NoTransparency` ignores transparency (everything is drawn opaque); `ScreenDoor` uses stipple patterns and needs no sorting; the `Add` and `Blend` families use additive or alpha blending, where the `Delayed` variants draw transparent objects after the opaque ones and the `SortedObject` variants also sort the transparent objects back to front. The initial value is whatever the Quarter widget starts with; read it with `transparencyType()`.

Public functions

Construction

`explicit QuarterExaminerViewer(QWidget* parent = nullptr)`

Creates the viewer: a `QuarterWidget` surrounded by the decoration. Quarter must already have been initialized with `Quarter::init()`, which in turn requires an existing `QApplication`.

`~QuarterExaminerViewer()`

Detaches the scene graph from Quarter and releases the viewer's references. The viewer must be destroyed before `Quarter::clean()` is called.

Scene graph and camera

`void setSceneGraph(SoNode* scene)`

Sets the scene to display. The viewer takes its own reference on `scene`, so the caller keeps (and must eventually release) its own. Any previous scene is released; passing `nullptr` clears the viewer.

If the scene contains no `SoCamera`, the viewer creates an `SoPerspectiveCamera`, places it ahead of the scene, fits it to the scene when the window is first shown and stores that view as the home position. If the scene already contains a camera, it is used unchanged and its current state is stored as the home position.

`SoNode* getSceneGraph() const`

Returns the scene passed to `setSceneGraph()`, or `nullptr`.

`SoCamera* getCamera() const`

Returns the camera in use (the viewer's own or the scene's), or `nullptr` if no scene is set. Use it to apply changes to the camera directly. The pointer changes when the camera type is switched, so do not cache it across `setCameraType()` / `toggleCameraType()`; connect to `cameraTypeChanged()` to be told when that happens. After modifying the camera yourself, call `viewAll()` and/or `saveHomePosition()` as needed.

`SIM::Coin3D::Quarter::QuarterWidget* quarterWidget() const`

Returns the wrapped Quarter widget for functionality not exposed here.

State queries

`bool isViewing() const`

True in View mode, false in Pick mode.

`bool isHeadlight() const`
True if the headlight is enabled.

`bool isDecoration() const`
True if the buttons and thumbwheels are shown.

`CameraType cameraType() const`
The current camera projection.

`DrawStyle drawStyle() const`
The current draw style.

`TransparencyType transparencyType() const`
The current transparency type. The value is read from the Quarter widget, so it stays correct if the widget was changed directly through `quarterWidget()`. If Quarter is set to a mode that has no `TransparencyType` equivalent, the last value set through `setTransparencyType()` is returned.

`QColor backgroundColor() const`
The current background color.

Seek settings and popup menu

`void setSeekTime(double seconds)`
Duration of the seek animation. 0 jumps immediately. Default 2.0.

`double seekTime() const`
Returns the seek time.

`void setSeekDistance(float percent)`
Percentage of the distance to the target that remains after a seek. Default 50.

`float seekDistance() const`
Returns the seek distance.

`void setSeekToObject(bool on)`
If true, seek to the center of the picked object's bounding box instead of the picked point. Default false.

`bool isSeekToObject() const`
Returns the seek target setting.

`void setPopupMenuEnabled(bool on)`
Enables or disables the right-click popup menu.

`bool isPopupMenuEnabled() const`
Returns whether the popup menu is enabled.

Public slots

`void viewAll()`
Fits the whole scene in the window. Does nothing without a camera.

`void saveHomePosition()`
Stores the current camera position, orientation, focal distance and view size as home.

`void resetToHomePosition()`
Restores the stored home viewpoint and cancels any seek animation.

`void seek()`
Arms seek: the next left click on the canvas picks a target and the camera flies to it. A click on the background does nothing.

`void setViewing(bool viewing)`
Selects View mode (true) or Pick mode (false). Updates the buttons and cursor and emits `viewingChanged()`.

`void setHeadlight(bool on)`
Enables or disables the headlight, which always shines along the viewing direction.

`void setDecoration(bool on)`
Shows or hides the buttons and thumbwheels.

`void setCameraType(CameraType type)`
Switches the projection by replacing the camera node in the scene graph, preserving position, orientation and apparent size. Emits `cameraTypeChanged()`. Does nothing if the camera cannot be replaced (for example if its parent is not a group node).

`void toggleCameraType()`
Switches between perspective and orthographic.

`void setDrawStyle(DrawStyle style)`
Sets the render style.

`void setTransparencyType(TransparencyType type)`
Sets how transparent objects are rendered. The change is applied to the Quarter widget immediately and is reflected in the popup menu and the preferences dialog. Use `Blend` for general use; use `SortedObjectBlend` when overlapping transparent objects must look correct from every direction, and `ScreenDoor` when sorting is too costly or alpha blending is not available.

`void setBackgroundColor(const QColor& color)`
Sets the background color.

`void showPreferences()`
Shows the modal preferences dialog.

Signals

`void viewingChanged(bool viewing)`
Emitted when the Pick/View mode changes.

`void cameraTypeChanged(QuarterExaminerViewer::CameraType type)`
Emitted after the camera node has been replaced. Call `getCamera()` again to obtain the new camera.

Protected functions

`bool eventFilter(QObject* obj, QEvent* ev) override`

Filters events of the Quarter canvas to implement mouse navigation, seek picking, the popup menu and the Esc key. Reimplement and call the base implementation to add custom handling.

`void showEvent(QShowEvent* ev) override`

Performs the initial fit of a viewer-created camera once the viewport size is known.

Notes

Lighting

The headlight is separate from the scene graph and always follows the camera. A light node inside the scene (for example the `SoDirectionalLight` in the Inventor Mentor "Hello, Cone" example) is fixed in world space and therefore appears to turn with the object when you orbit. Omit scene lights and use the headlight if you want the light to follow the view.

Camera ownership

A camera created by the viewer is part of the viewer's private root node, not of your scene graph; your scene is not modified. A camera that is part of your scene is used and modified in place.

Lifetime

The viewer holds references on the scene and on its private root. Destroy the viewer before calling `Quarter::clean()`, and release your own scene reference after the viewer is gone or at any time after `setSceneGraph()` if you no longer need the scene.

Differences from SoQtExaminerViewer

- No spin animation after releasing a drag.
- No stereo viewing, axes cross or help/set-up-view menu items.
- Navigation is implemented by the viewer, not by Quarter's scxml navigation systems; Quarter's own navigation is bypassed in View mode.
- Draw styles and transparency types are those provided by Quarter.

Files

File	Contents
QuarterExaminerViewer.h / .cpp	The viewer class.
ThumbWheel.h / .cpp	The thumbwheel widget used by the decoration (internal).
main.cpp	"Hello, Cone" test program.
CMakeLists.txt	Builds the library `quarterexaminerviewer` and `hello_cone`.

See also

Quarter (QuarterWidget), Coin (SoCamera, SoRenderManager), The Inventor Mentor, `SoQtExaminerViewer`.